Operating Systems Technologies

(Tecnologias de Sistemas Operativos)

Introduction

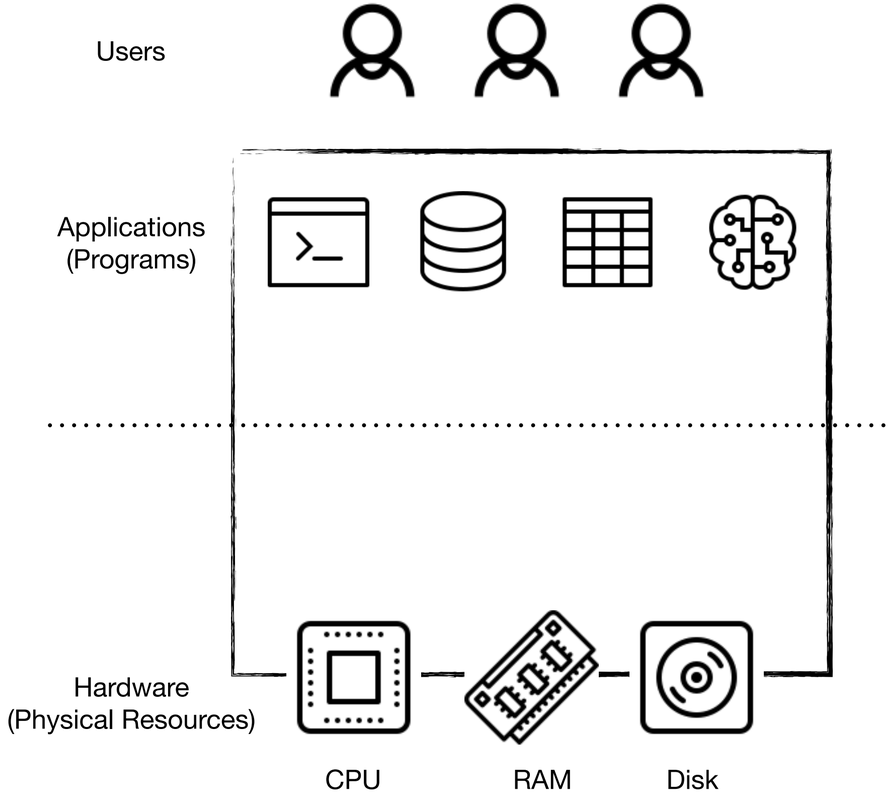
University of Minho

2025-2026

The road so far…

Landscape

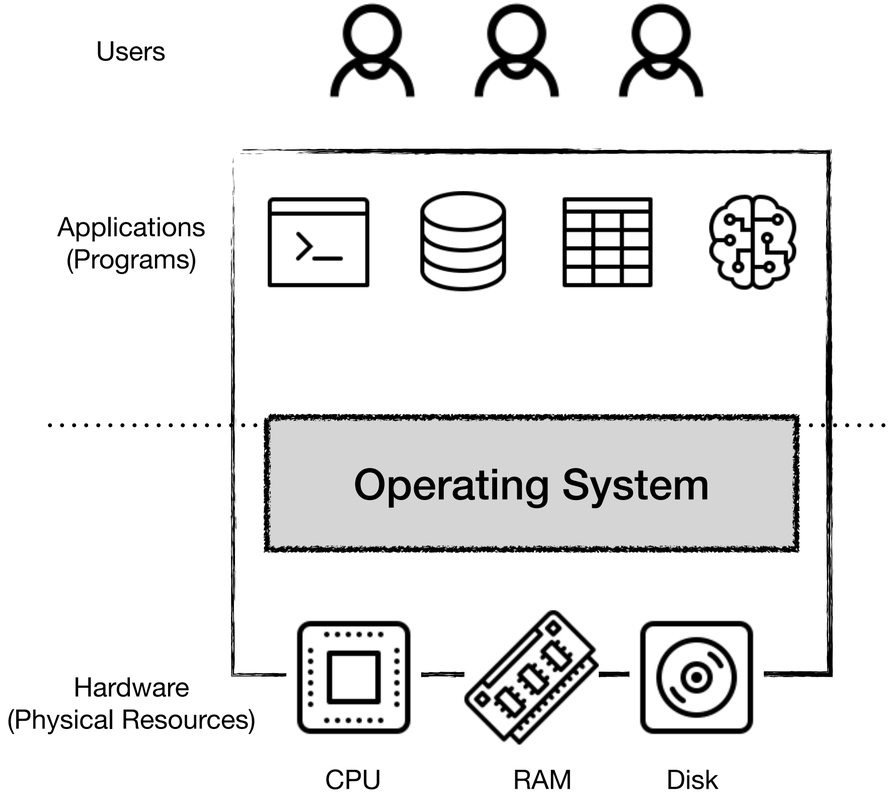
- Applications (Software)
  - Programs executing millions/billions of instructions per second (*e.g.*, databases, AI pipelines)
- Physical resources (Hardware)
  - CPU: where instructions are executed
  - Memory: stores volatile data structures
  - I/O Devices: external devices to interact with
    - Disk: to store data persistently
    - Keyboard, mouse, display, printer, …



The road so far…

Enters the Operating System (OS)

- Mediates the interaction between programs and hardware
- **Abstracts** physical resources into general, powerful, and easier-to-use virtual forms
- Offers **interfaces** (APIs) to use these resources (e.g., run programs, use memory and devices)
- **Manages** resources for multiple programs (e.g., concurrent programs, resource sharing)



First, the good news!

Do you remember the OS course?

- You already know the fundamentals of Operating Systems
- CPU Management
  - The process abstraction and API
  - CPU scheduling
- Memory Management
  - Address space management
  - Extending physical memory
- Persistence
  - I/O devices (HDDs, SSDs)
  - File Systems interface and design

Now, even better news…

If you wish to learn more OS stuff!

- The landscape has been evolving rapidly
  - new hardware (GPUs, CPUs, network, disks)
  - new technologies and libraries (eBPF, IO_URing, SPDK, …)
  - new assumptions (safe OS extensions, user-space designs, asynchronous and polled I/O, …)
- However, most applications and systems are still designed for traditional hardware and OSs!

TensorFlow

PyTorch

PostgreSQL

MySQL

APACHE Spark

Why Should I Do This Course?

- Most developers lack knowledge about modern technologies and how to properly debug the interaction of their applications/systems with the OS!
- Such knowledge is crucial for building better systems and applications
  - more performant, efficient, and resilient

libfuse/**libfuse**

The reference implementation of the Linux FUSE (Filesystem in Userspace) interface

208 Contributors | 4 Used by | 76 Discussions | 6k Stars | 1k Forks

Roadmap

Theory

- Introduction
  - TSO presentation & OS recap
- Understanding our systems
  - Observability
  - Benchmarking
- CPU Scheduling
  - Fair scheduling & multi-CPU scheduling
- Storage and Memory
  - “User-space” file system design
  - Caching, page cache (design and eviction), and data durability
- I/O Optimizations and Libraries
  - Compression, deduplication, and encryption
  - Modern I/O libraries (libaio, IO_URING, and SPDK)

Practice

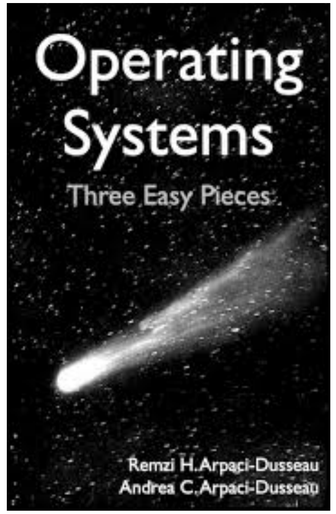
- Warmup
  - VM and lab environment set up
- Observability and Benchmarking
  - top, strace, perf, eBPFs
  - stress-ng, FIO
- Scheduling
  - eBPF-based schedulers
- Memory and Storage Management
  - FUSE
  - Custom page cache
- I/O Optimizations and Libraries
  - Data compression/encryption
  - IO_URING

Assessment

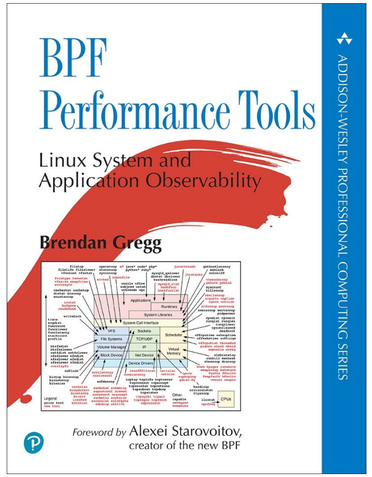
- <u>Project (50%)</u>
  - Report: 17/05/2026
  - Presentation: 01/06/2026 - 05/06/2026
- <u>Written exam (50%) – minimum grade: 8 values</u>
  - 25/05/2026

Main References

- Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau.
  Operating Systems: Three Easy Pieces. Arpaci-Dusseau Books, 2018.



- Brendan Gregg.
  BPF Performance Tools. Addison-Wesley, 2020.



Team

João Paulo
jtpaulo@di.uminho.pt

Tânia Esteves
d12729@di.uminho.pt

Questions?